Python

Programming

- Python Programming

  - It is widely used general-purpose, high-level programming language
  - It was created by Guido Van Rossum in 1991

- Features:
  - Open Source
  - High-level lang.
  - Simple
  - Platform Independent
  - Dynamically Typed
  - Compiled & Interpreted
  - OOP that supports procedural, object-oriented & functional Prog.

- It is used for
  - Web Development (Server side)
  - S/w develop.
  - System Scripting
  - handle big data
  - analysis & analytics

- Diff. Python vs
  Java vs
  C++

C++	Java	Python
Compiled, Prog. language	Compiled	Interpreted
Supports Operator overloading	Does not support	Support
Provide both Single & multiple inheritance	Provide Partial multiple inheri	Provide both.
Platform dependent	Platform Independent	Platform Independent
Fast	Slow	Slow

- Modules & PIP

- Modules is one files that contains fn, classes & variables

- Pip is a package manager that can be used to install & manage modules

Pip install numpy
pandas

- Types of modules

① built in modules
② External

① Built-in module
- These modules are ready to import & use with the python interpreter
- No need to install.
- eg. random, enum, html, math

② External
- These modules are imported from a third party files.
- Install using package manager like pip or conda.

pip install module name

- eg. pandas, numpy, sklearn, scikit learn, matplotlib, seaborn, django, etc.

- Tut 1 : First Program.

- Tut 2

- Escape sequence char → \n

- Comment → # → "single line"
''' ''' → "Multi line"

- Double Quote character → \" \"

  I am \" Ankit \"

  → I am "Ankit".

- Tut 3 - Variables & Data types

  (i) Number : int, float, complex
  (ii) str
  (iii) Bool
  (iv) list
  (v) tuple
  (vi) dict

- Tut 4 - Calculator Program

- Tut 5 - Typecasting in Python

  - Conversion of one data type to other

  Types :

  (i) Implicit
  (ii) Explicit

```
a = "5"
print (int (a))
```

① Explicit
- Conversion done by developer. Programmer or manually

② Implicit
- Python interpreter automatically convert.

- Tut 6 : Take a input

```
a = Input ("Ente 1 no : ")
b = Input ("Ente 2 no : ")

print (int(a) + int(b))
```

- Strings (tut 7)

- String Methods (tut 8)

- Capitalize()
- Center()
- Count()
- Find()
- endswith()
- Formad()
- index()
- isalnum() : string cha alpha numerical a not
- isalpha() : alphabet
- isdecimal()
- isdigit()
- islower()
- isnumeric()
- isspace()
- istitle()
- isupper()
- join()

- lower()
- lstrip() - Returns the string with
- replace()
- rstrip()
- slice()
- Swapcase() - convert all uppercase to lowercase & vice versa
- title()
- upper()
- zfill() : Returns a copy of the string with '0' character padded to left side of string

- Task 9 : Conditional statement

if else

```
a = 5
if (a<5):
    Print ("  ")
else :
    Print (" ")
```

if-elif-else

```
if () :
    Print ()
elif () :
    Print ()
else :
    Print ()
```

- Python Version testing

```
import os
Print ("Hello...")
os.system ("python --version")
```

- Tut 10 : Match statements

```
x = 4
match x :
    case 0 :
        Print ();
    case 4 :
        Print ();
    case _ :
        Print ();
```

- Tut 11 : For loops

```
name = "Abhishek"
for i in name :
    Print (i)
```

```
for k in range (1, 9):
    print (k)
```

- Tut 12 : While Loops.

```
i = 0
while (i<=4):
    print (i)
    i = i+1
```

- Tut 13 : Break & Continue

Break : used to terminate loop :

Continue : it execute the next iteration of loop .

Pass : used when a statement is required syntactically but you do not want any command or code to execute

```
for i in range (12):
    if (cond^n):
        break
    # statement
# loop end
```

• Tw 14 : functions

- A fn is a block of code that performs a specific task whenever it is called

• Types.

① Built in fn.

- Pre-defined & pre-coded

eg. min(), max(), len(), sum(), dict(), list(), tuple()

② User defined fn

- We can create fn to perform specific tasks as per our needs.

```
def sum (a, b):
    add = a+b
    print (add)

a = 5;
b = 8 :
sum (a, b)
```

Page No.
Date

⊙ Two is : fn Arguments

① Default Argu

- It assumes default value if a value is not provided in fn call.

```
def fun (a, b):
    print (a+b)

fun (4, 5)
fun (5)
```

② Keyword argu

- We can provide arg. with key + value so interpreter recognizes the argu. by parameter name.
- No need of order in argument passed.

```
def fun (a, b):
    print (a+b)

fun (4, 1)
fun (b=5, a=4)
```

(iii) Required argu

- We don't pass arg with key + value, then it necessary to pass the argu in correct order & the no. of argu passed match with fn definition.

```
def avg (a, b, c=1):
    Print ((a+b+c)/3)
avg (4, 6)
```

(iv) Variable length argu.

(v) Arbitrary argu

- While creating a fn, pass a* before the parameter name while defining the fn.
- The fn accesses the arg. by processing them in tuple.

(vi) Keyword Arbitrary argu

- While creating fn, pass a * before parameter name.
- The fn accesses arg. by processing them in the form of dictionary.

(vii) Return Statement

- It is used to the value of the expression back to the calling fn.

```
def name ( fname, mname, lname ):
    return ( fname + " " + mname
             + " " + lname )
print (name ("Ankit", "Ashok", "Pawar"))
```

- Lit 16 : List in Python.

- list used to store multiple items in single variable

```
l = [1, 2, "AP"]
print [l]
```

List methods :

(i) append ()
(ii) copy ()
(iii) clear ()
(iv) count ()
(v) extend ()
(vi) index ()
(vii) insert ()
(viii) pop ()
(ix) remove ()
(x) reverse ()
(xi) sort ()

Tut 17 : Tuples in Python

- Tuples are ordered collection of data items.
- They store multiple items in single variable.
- Tuples are unchangeable meaning we can not alter them after creation.

eg.

```
tup = (1, 2, 3, "AP")
Print (tup)
```

Operation :

(i) Traversing
(ii) Concatenation
(iii) Nesting
(iv) Repetion
(v) Slicing
(vi) Finding length
(vii) Index
(viii) Count.

Tut 18 : f-string & docstrings

- f-string - representing visual data

s = "I am {} & I am {}"

a = "I am {} & I am {}"

Print (f "I am {name} & I am {lang}"

name = "Ankit"

lang = "Engi".

- Docstrings

```
def square (n):
    ''' ghari jara '''
    print (n**2)

square (5)
print ( square. __doc__)
```

- It is the string literal that appear right after the def of a fn, method, class, or module.

Tut 19 : Recursion in Python

- Recursion is the process of defining something in terms of itself.

- It is a programming concept when a f^n calls itself either directly or indirectly.

```
def fact(n):
    if (n==0 or n==1):
        return 1
    else:
        return n * fact(n-1)

print (fact(3))
```

Tut 20 : Sets in Python

- Used to stored multiple items in single variable
- It is a collection which is unordered, unchangeable & unindexed
- It is unchangeable but you remove & add new items

```
st = {"AP", 2, 3, "banana"}
Print (st)
```

Page No.
Date

- Methods

(i) add()
(ii) clear()
(iii) copy()
(iv) difference()
(v) difference_update()
(vi) intersection()
(vii) intersection_update()
(viii) isdisjoint()
(ix) issubset() (x) issuperset()
(xi) pop()
(xii) remove()
(xiii) symmetric_diff()
(xiv) union()
(xv) update()

- Tut 22: Dictionaries in Python

- It is data structure that stores data in key - value pairs

- It is collection of ordered, changeable & do not allow duplicates

```
dic = {
    544 : "Harry",
    "name" : "AP"
}
print (dic)
```

(i) clear() (ii) copy() (iii) get()
(iv) items() (v) keys() (vi) pop()
(vii) popitem() (viii) update() (ix) values()

Twt 22 : for loop with else in Python

Twt 23 : Exception Handling in Python

Diff types of exception :
(i) Syntax Error (ii) Type Error (iii) Name Error
(iv) Index Error (v) Key Error (vi) Value Error
(vii) Attribute Error (viii) IO Error.

- Try & Except statements

- Try & Except statements are used to catch & handle exception in Python.

Twt 24 : Finally keyword in Python

- Finally block always executed after normal termination of try block.

Tut 25 : Raising Custom Error

```
a = int (input ("Enter val bet^ 5 & 5"))
if (a<5 or a>5):
    raise ValueError ("Value Error")
```

Tut 26 : Short hand if else statement

. If . Else in one line

```
a = 330
b = 3303
print ("A") if a>b else print ("=")
        if a==b else print ("B")

a = 2
b = 330
print ("A") if a>b else Print ("B")
```

Tut 27 : Enumerate fn

- It takes a collection & return it as an enumerate obj

- It adds a counter as the key of enumerate obj

```
fruits = ["apple", "banana", "mango"]
for index, fruit in enumerate(fruits):
    print(index, fruit)
```

- Python linter : It is a tool that analyzes python source code to identify & correct issues like : Syntax error, Bugs, Improper Indentation.

Some common Python linters include
- Pylint
- Mypy
- Black
- ISort
- Ruff.

Tut 28 : import use in python.

```
import math
result = math.sqrt(9)
print(result)
```

- from keyword.

```
from math import sqrt, pi

res = sqrt(9) + pi
print(res)

// for all functionality in module use
    from math import *
```

- as keyword

```
import math as m

ans = m.sqrt(9) + m.pi
print(ans)
```

- dir fn.

```
import math
print(dir(math))
```

- Tut 29 : If __name__ == "__main__" in python

- main.py :

```
import ankit
ankit.Welcome()
```

- ankit.py :

```
def Welcome():
    Print("Hey welcome!")

if __name__ == "__main__":
    Welcome()
```

- Virtual Environment :

It is an isolated space where you can work on your py. Projects seperately from your system - installed Python.

- install virtual env.

```
pip install virtualenv
```

- Ter

```
virtual env --version
```

- Create

```
Python -m venv myenv
```

- Activation

```
myenv \ Scripts \ activate.bat
```

- Deactivation

```
deactivate
```

- Tut 80 : OS module

- It is a built-in library that provides fn for interacting with the OS.

fn :

- os.name
- os.getcwd()
- os.listdir()
- os.mkdir()
- os.rename
- os.environ
- os.exit()
- os.abort()

```
import os

if (not os.path.exists ("data")):
    os.mkdir("data")

for i in range (0, 100):
    os.mkdir (f"data/Day{i+1}")
```

```
import os

for i in range (0, 100):
    os.rename (f"data/Day{i+1}",
               f"data/Tutorial{i+1}")
```

Twl 31 : local vs Global variable

- local - inside fn or block
  - exists only during fn execution.

- Global :- outside
  - It remains in memory for the duration of program

```
x = 10

def my_fun():
    global x
    x = 4
    y = 5
    print(y)

my_from()
    print(x)
# print(y)
```

Twl 32 : FILE IO in Python

It is the process of performing op^n on file such as creation, read, write

modes :

(i) read (r)
(ii) write (w)
(iii) append (a)
(iv) create (x)
(v) text
(vi) binary (rb)

① Methods : Tut 33

(i) close()
(ii) fileno()
(iii) read()
(iv) readable()
(v) readline() : line by line
(vi) seek() : change file position
(vii) tell() : current position returns
(viii) truncate() : Resizes the file to specified size
(ix) writable()
(x) write()
(xi) writelines()

② Lambda fn : Tut 34.

- It is fn without a name that can be used to evaluate & return a single expression

lambda parameter : expr

```
# def double (x):
#     return x*2

double = lambda x: x*2
print (double(5))
```

Map, filter & reduce : TW 35

Map :

```
def cube (x):
    return x*x*x

l = [1,2,3,4]

newl = list (map (cube, l))
Print (newl)
```

OR

```
l = [1,2,3,4]

newl = list (map (lambda x: x*x*x,
                  l))
Print (newl)
```

Filter:

```
l = [1,2,3,4]
def filter_fun(a):
    return a>2
newl = list(filter(filter_fun, l))
print(newl)
```

Reduce:

```
from functools import reduce
l = [1,2,3,4]
def myfun(x,y):
    return x+y
sum = reduce(myfun, l)
print(sum)
```

'is' vs '==' in Python Tut 36

```
a = 4
b = 4
```

Print (a is b) # exact location of obj in memory

Print (a == b) # value.

OOP in Python.

It is used for enhanced code structure, code reusability, better security, better maintainability & flexibility

Classes & obj (tut 37)

```
class Person :
    name = "AP"
    occ : "SDE"
    network : 100000
    def info (self) :
        print (f"{self.name} is a {self.occ}")

a = Person()
a.info()
```

Constructor (tut 38)

```
def __init__ (self) :
```

Page No.
Date

- self refers to current instance of class.
- `__init__` is automatically called when a new obj is instantiated using a class
- Parameterized constructor :

```
class Person:
    def __init__(self, n, a):
        self.name = n
        self.age = a

    def info(self):
        print(f"{self.name} is {self.age}")

a = Person("AP", "JDE")
b = Person("PL", "SDE")
a.info()
b.info()
```

- Default constructor

```
class Person:

    def __init__(self):
        print("Hello")
```

Decorators in Python (Lvl 89)

- Modify or extend the behavior of fn / methods, without changing their actual code

```
def greet (fx):
    def mfx():
        print ("Good mornings")
        f(x)
        print ("Tanks")
        return mfx

@ greet

def hello ():
    print ("Hello")

# greet (hello) ()
hello()
```

OR

```
def greet (fx):
    def mfx (*args, **kwargs):
        print ("GM")
        fx (*args, **kwargs)
        print ("Thxn")
        return mfx
```

@ greet

```
def add (a,b)
    print (a+b)

greet (add) (1,2)
```

- Getter & Setter (tut 40)

Getter : A method that allows you to access an attribute in given class

Setter : A method that allows you to set or mutate the value of an attribute ~~of~~ in class.

- It (both) encapsulate the fields of class.

- Inheritance (tut 41)

```
class BaseClass:
    Body
class derreClass (BaseClass):
    Body
```

Access Modifier / Specifier (tut 42)

- A fundamental concept in OOP that allow us to control the visibility & accessibility of attr & methods within class

- Public : Everyone can access
- Privat : only accessed within same class
- Protected : accessed in class & its subclass.

Static Methods (tut 43)

- It is a method that is associated with class, not obj.

```
class math:
    @staticmethod
    def add(a, b):
        return a+b

a = Math.add(1,2)
print(a).
```

- Instance vs Class Variables (tut 44)

  - Instance variable is variable unique to each obj created from class.

  - Class variable is variable shared across all instances of class.

- Class Methods (tut 45)

  - It is built in fn in Python that used to define a method that is bound to class & not the obj.

- class methods vs Alternative Constructors (tut 46)

  - It allowing you to create obj instance of class using diff I/P formats or data source.

- dir, __dict__ & help method (tut 47)

  - dir() : It returns a list of all attributes & methods available for an object.

- dict() : It returns a dictionary representation of an obj attributes.
- help() : It is used to get help documentation for an obj, including a description of its attributes & methods.

Super keyword (tut 48)

- It is a built-in fn that allows access to methods & Prop. of a parent from a child class

Magic / Dunder methods in Python (tut 49)

- Python Magic methods are the methods starting & ending with double underscores '__'. They are defined by built-in classes in Python & commonly used for operator overloading
- also called "Dunder Methods"
- means "Double Underscores"

eg: __init__ (initialization)
__del__
(Arithmetic) → __add__, __sub__,
(string) → __str__, __repr__, __dir__

(read in documentation)

- Method Overriding (tut 50).
  - It refers to defining a method in a subclass with the same name as a method in the superclass.
- Operator Overloading (tut 51)
  - It allow the same operator name or symbol to be used for multiple operations
- Inheritance (tut 52)
  - Single Inheritance.
  - Multiple —"—.
  - Multilevel —"—.
  - Hierarchical —"—.
  - Hybrid —"—.
- Time Module (tut 53)
  - It provides various fn for working with time, including parsing, program execution & retrieving the current time
  - eg time.ctime() time.localtime(), time.strftime(), time.sleep().

- Walrus Operede (tut 55)

':=' that assigns values to variable as part of larger expression

- Shutil Modul (tut 56)

- It allows user to perform high level file op^n, such as copying, creating, & removing files & directories.

Copy() . copy2(), copytree(), move()

- Requests Module (tut 57)

- It allows you to send HTTP requests using python.

- The HTTP request returns a response obj with all response data.

- Beautiful Soup

- It is a free, open source python library that helps you extract data from HTML & XML doc.

Generators (tut 58)

- It is a type of iterable like lists or tuples, but unlike lists, they do not store their contents in memory.

- It is a fn or exp that will produce a given iterable one obj at time, on demand.

function Caching (tut 59)

- It store the results of expensive fn calls & reuse them when the fn is called with same arg. again

- Memoization

- It save the results of fn calls in comp memory
- When fn is called again with same arg, the stored result is return instead of recalculating it

Regular Expression (tut 60)

It is a special seq. of characters that uses a secrd pattern to find a string or set of strings.

fn: re.findall(), re.split(), re.sub()
re.search(), re.compile(), re.escape()

- Metacharacter (see in GFG)

Asyncio (tut 61)

It is python library that allows developers to write concurrent code using the async - await syntax

Multithreading (tut 62)

It allows concurrent execution of multiple threads within a singh process

Threads : light weight process
- seq. of instructions that can run independently of rest of program

- concurrent features.

- Multiprocessing (Vid 63)

  - It is built-in package that allows a system to run multiple process simultaneously.

- Creating Command line Utility (Vid 54)

  It ~~use~~ allows users to perform tasks by typing commands into terminal or Console

Jupyter Notebook

- Install
  - cmd
  - pip install jupyter
- Run
  - cmd
  - jupyter notebook
- Install Python libraries
  - !pip install numpy

(! in jupyter ~~on~~ notebook use)

NumPy

NumPy - Numerical Python

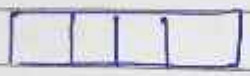
- Numpy install :

  ! pip install numpy

- print (np.--version--) (Shift + Enter to run)

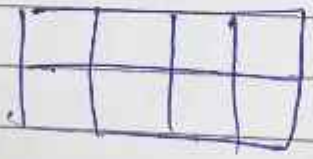
1 Creating Arrays

① 1D Array



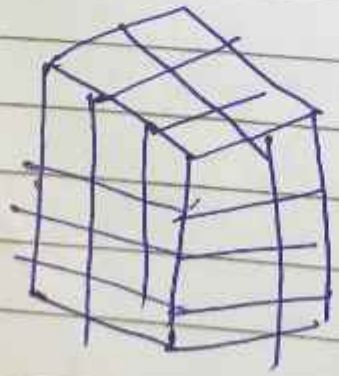
Shape : (4)

② 2D Array



Shape : (2, 4)

③ 3D Array



Shape (4, 3, 2)

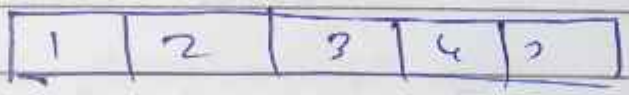
2 numpy array attributes

3 Array initialization methods

1. Zero Array - np.zeros
2. Ones Array - ones
3. Full Array - full ((2,3),7)
4. Identity Array - eye (3)
5. Empty Array - empty (2)
6. Evenly spaced Array - arrange (1,3,1)
7. Specific no. of equally (spaced) values b/w a range - linespace (1,11,7)
8. Random values array - float np.random.rand(3,4)
9. Random value array - int - np.random.randint (3,4)

4 Array Indexing & Slicing

- Indexing



+ve index 0 1 2 3 4

-ve index -5 -4 -3 -2 -1

- Access individual characters in a string using their index

· Slicing

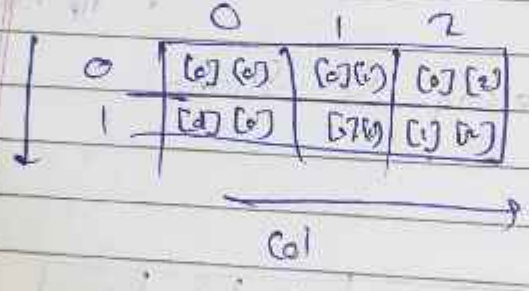
A feature that enables accessing parts of sequence

Syntax : array_name [start : stop : step]

	M	A	D	H	V
index	0	1	2	3	4

name [0]
= 'M'

2d array slicing



5 Array Reshaping & flattening

The reshape () fn allows you to change the dimensions of an array without altering its data

Flattening transforms a multi-dimensional array into a one-dimen array

6 Array Stacking & Splitting

- Stacking f^ combine array along a new or existing axis

- Splitting f^ divide a single array into multiple smaller sub arrays. The resulting sub-arrays are returned as list.

7. Mathematics Operations on Array

9 Statistical functions

10 Array Comparison

11 Broadcasting

- A powerful mechanism that allows arithmetic operation on array of diff^ shapes & sizes by automatically "stretching" the smaller one to match the shape of larger one.

12 Handling with nan & inf

Save & load : Array.

Python Pandas

It is an open source library for data analysis & manipulation

| Pip install pandas (Jupyter)

1. DataFrame
It is a 2D table like structure in python where data is arranged in rows & columns

2. Basic Dataframe Understanding

3. Save & load Data from CSV

4. Rows & Colm · Selection

5. Filter Dataframe

6. Row & Cols : Operation (add, update, delete)

7. Working with date values

8. Handling missing values

9. Aggregation & group by.

10. Concatenate & Merge Dataframes (Joins)

AI & chatGPT

Matplotlib lib.

It is comprehensive, open-source lib for the Python prog. lan. used to create static, animated, & interactive visualizat?

import matplotlib.pyplot as plt

1. Basic Example.

2. Pyplot API

- It is collection of command style fn in Matplotlib lib that provides a MATLAB-like interface for creating plots & visualizat?

- Univariate - The simplest form of quantative data analysis, focusing on describing, summarizing & identifying patterns within a single variable at a time

3. Lineplot.

- Expand more parameter. (cont)

4. Histogram

5 Boxplot

6 Piechart

7. Camplot

8 Bivariate Analysis : num : num

9 Scatter plot

10 Barchart

11 # # Bivariate : num : cat

12. Multivariate : num : num : num

13 Bubble plot.

14 Multi : 2 num : 1 cat

15 Object Oriented API

16 3D Plot

Seaborn lib

Page No.
Date

- It is powerful open-source python lib for creating attractive & informative statistical graphics